SOFTWARE DESIGN

Sequence Diagrams, API Design, Error Handling and UX

Project: Smart Expense & Budget Management Application

Group Number: G-15

Authors:
Rhea Sushanth – PES1UG24CS371
Sarthki Vachhani – PES1UG24CS422
Sai Manogna BV – PES1UG24CS399

Part B: Design

4. Design

4.1 Design Overview

The design realizes the layered architecture of Section 3 as concrete interactions, interfaces and data structures. It is organized as four interaction flows that carry the most risk (4.2), the full HTTP API contract (4.3), a single consistent approach to error handling, logging and monitoring (4.4), and the UX, open issues and supporting class/decision material (4.5-4.6, Section 5).

4.2 UML Sequence Diagrams

Four sequence diagrams describe the interactions that carry the most risk: login, transaction entry with alerting, CSV import, and scheduled generation.

Login (UC-01, FR-002, SEC-004, SEC-006)

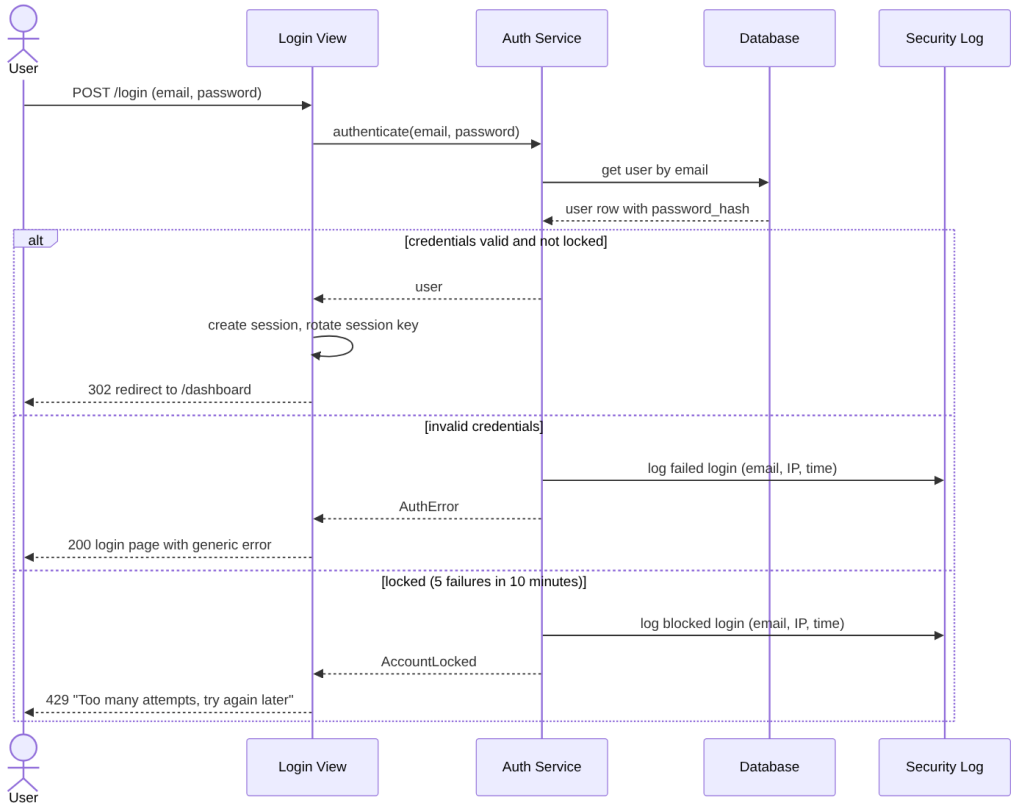


Figure 4: Login sequence diagram

Add expense and evaluate alerts (UC-02, UC-07, FR-005, FR-007, FR-013, FR-015, FR-016)

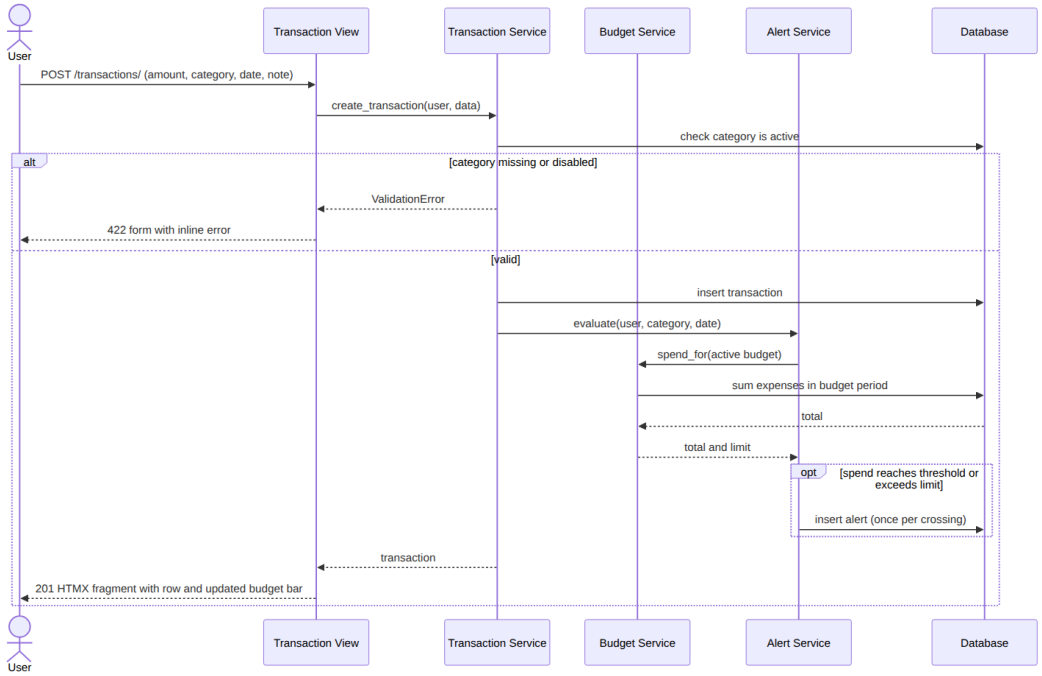


Figure 5: Add expense and evaluate alerts sequence diagram

Note: Alert Service `evaluate(user, category, date)` looks up the budgets of that category whose period contains the transaction date and calls Budget Service `spend_for()` for each (Section 3.4).

CSV import (UC-05, FR-009 to FR-011, NFR-004, SEC-003)

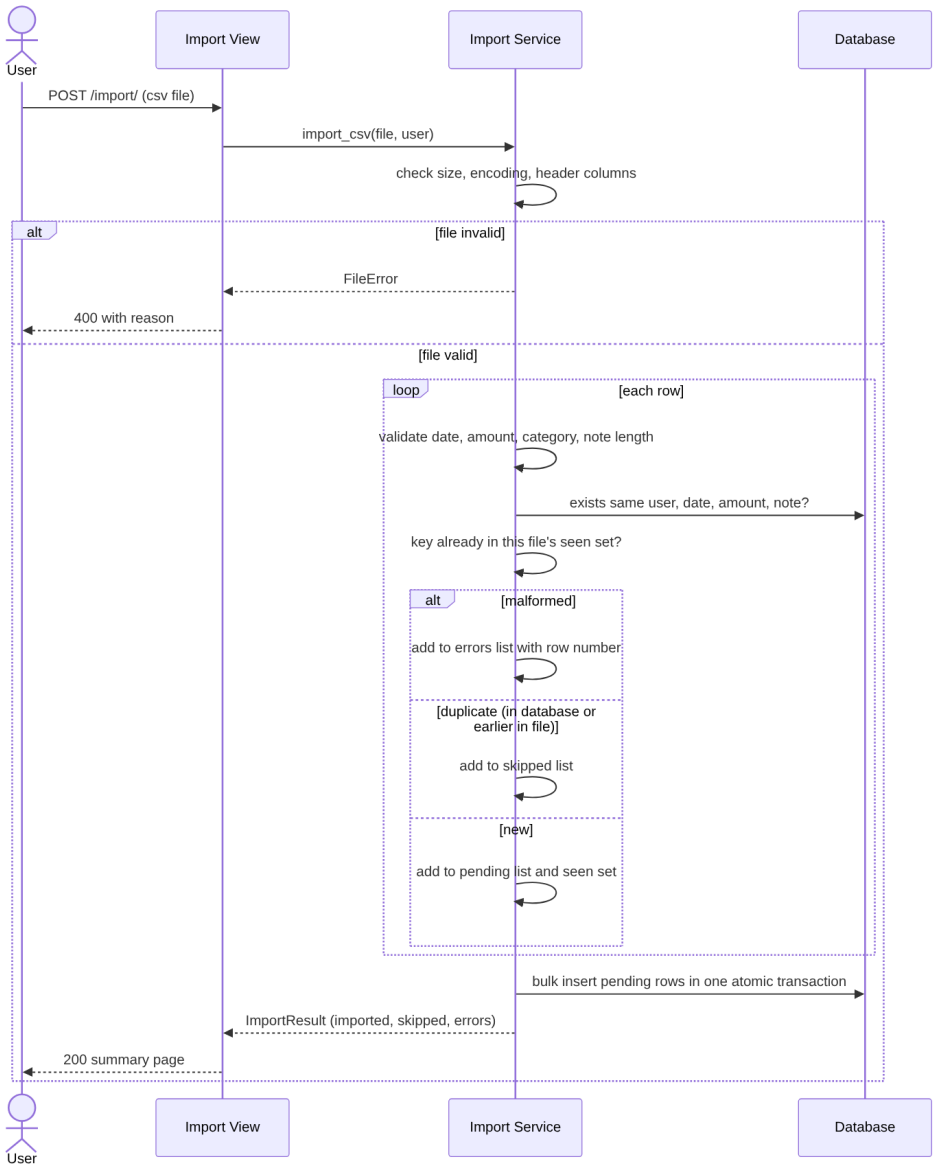


Figure 6: CSV import sequence diagram

A row that repeats an earlier row of the same file is caught by the seen set and skipped as a duplicate, so it never reaches the bulk insert and cannot roll back the whole import (FR-010, FR-011). Because a bulk insert does not fire the post-save alert hook, the Import Service calls Alert Service `evaluate(user, category, date)` once for each category and date in the imported rows after the insert commits; the sandbox sync does the same (FR-015, FR-016).

Scheduled recurring generation (UC-11, FR-024)

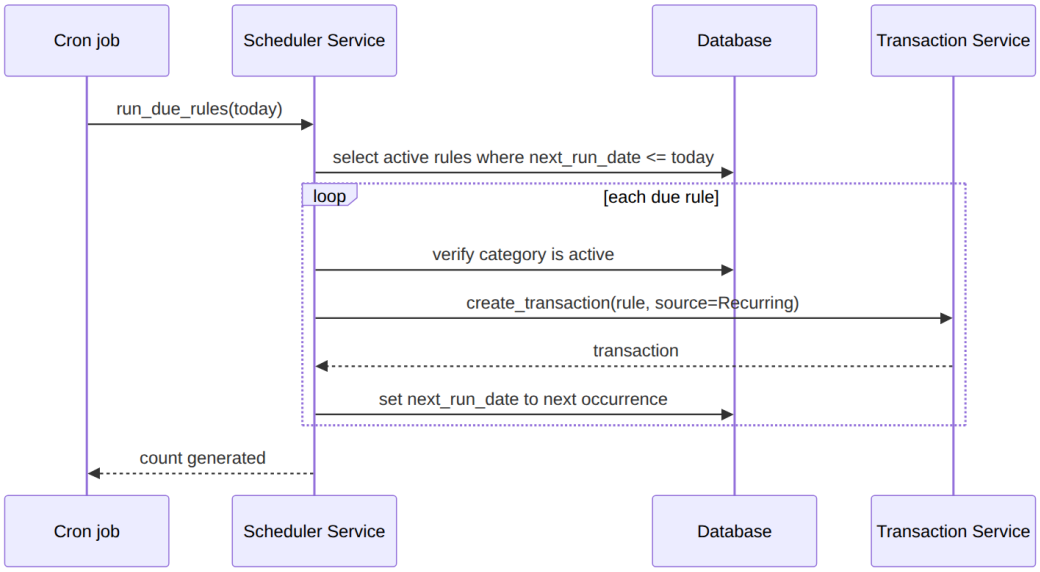


Figure 7: Scheduled recurring generation sequence diagram

4.3 API Design

The application exposes HTTP endpoints of three kinds: full HTML pages, HTMX fragments (when the request carries the `HX-Request` header), and JSON for chart data only. All endpoints except register and login require a valid session; all state-changing requests are POST (or DELETE) with a CSRF token. Interfaces are shown here for at least two components (Auth and Transactions), plus the remaining components for completeness.

Conventions

Item	Rule
Base path	`/` on one host, HTTPS only
Authentication	Django session cookie; unauthenticated requests get 302 to `/accounts/login/` (or 401 for JSON)
CSRF	`X-CSRFToken` header or `csrfmiddlewaretoken` form field on every POST and DELETE
Ownership	Every resource lookup is filtered by the current user; other users' IDs always return 404
Dates and money	ISO dates `YYYY-MM-DD`; amounts as decimal strings with 2 places, for example `"250.00"`
Success codes	200 read or update, 201 created, 302 redirect after a full-page form post
Error codes	400 malformed request or file, 401 not authenticated (JSON), 403 not permitted, 404 not found or not owned, 422 validation failure, 429 account temporarily locked, 500 server error

Auth API (accounts)

Method and path	Role	Input	Success response	Errors
`POST /accounts/register/`	Public	`name`, `email`, `password` (at least 8 characters, one letter and one digit)	302 to dashboard	422 duplicate email, password below the policy

Method and path	Role	Input	Success response	Errors
POST /accounts/login/	Public	email, password	302 to dashboard	200 login page with generic error; 429 when locked
POST /accounts/logout/	User	none	302 to login, session destroyed	403 missing CSRF
GET /accounts/profile/	User	none	200 profile form	302 not logged in
POST /accounts/profile/	User	name	200 updated profile	422 empty name

Transaction API (transactions)

Method and path	Role	Input	Success response	Errors
GET /transactions/	User	optional from, to, category, type	200 list page or fragment	422 bad date range
POST /transactions/	User	type, amount, category, date, note	201 fragment with the new row and updated budget bar	422 missing category, amount not above 0, disabled category, category type does not match
POST /transactions/{id}/edit/	Owner	same fields	200 updated row	404 not owned; 422
DELETE /transactions/{id}/	Owner	none	200 empty fragment	404 not owned

Import API (transactions)

Method and path	Role	Input	Success response	Errors
POST /transactions/import/	User	multipart file (CSV, 2 MB max; columns date,amount,type,category,note; layout in SRS Appendix B.8)	200 summary: imported, skipped, errors[]	400 wrong header, not UTF-8, too large; row errors are reported, not fatal
POST /transactions/sync/	User (feature flag on)	none	200 summary of synced transactions	503 sandbox unavailable

Budget and alert API (budgets)

Method and path	Role	Input	Success response	Errors
GET /budgets/	User	none	200 budgets with limit, spent, remaining, percent	none
POST /budgets/	User	category, period, start_date, limit_amount, threshold_pct	201 new budget	422 overlap, disabled or Income category, limit not above 0, threshold outside 1 to 100

Method and path	Role	Input	Success response	Errors
POST /budgets/{id}/edit/	Owner	period, start_date, limit_amount, threshold_pct	200 updated budget	404 not owned; 422 overlap with another budget, invalid values
DELETE /budgets/{id}/	Owner	none	200 empty fragment (its alerts are removed with it)	404 not owned
GET /budgets/{id}/status/	Owner	none	200 fragment with spend bar	404
GET /alerts/	User	none	200 alerts, newest first	none
POST /alerts/{id}/read/	Owner	none	200 fragment	404

Report API (reports)

Method and path	Role	Input	Success response	Errors
GET /reports/category/	User	from, to	200 category totals with percent of total	422 from after to
GET /reports/budgets/	User	date (optional)	200 limit, spent, remaining per active budget	none
GET /reports/chart-data/	User	kind (pie, bar, line), from, to, group (day, week)	200 JSON, see example payloads below	400 unknown kind; 422 bad range

Recurring API (recurring)

Method and path	Role	Input	Success response	Errors
GET /recurring/	User	none	200 rules with next run date	none
POST /recurring/	User	amount, category, frequency, start_date (stored as the first next_run_date)	201 new rule	422 disabled category, amount not above 0
POST /recurring/{id}/edit/	Owner	amount, category, frequency, next_run_date	200 updated rule	404 not owned; 422 disabled category, amount not above 0
POST /recurring/{id}/deactivate/	Owner	none	200 rule inactive	404

Admin API (categories)

Method and path	Role	Input	Success response	Errors
GET /admin/categories/	Admin	none	200 category list	403 Customer
POST /admin/categories/	Admin	name, type	201 new category	403; 422 duplicate name

Method and path	Role	Input	Success response	Errors
POST /admin/categories/{id}/edit/	Admin	name	200 updated	403; 404
POST /admin/categories/{id}/disable/	Admin	none	200 is_active false	403; 404
POST /admin/categories/{id}/enable/	Admin	none	200 is_active true	403; 404
GET /admin/users/	Admin	none	200 user list without password data	403 Customer

Example payloads

Create transaction (POST /transactions/; form fields shown as JSON for readability):

```
{ "type": "Expense", "amount": "250.00", "category": 3, "date": "2026-09-26", "note":
"Lunch" }
```

Success (201) returns an HTML fragment; the equivalent data is:

```
{ "txn_id": 118, "type": "Expense", "amount": "250.00", "category": "Food", "date": "2026-
09-26", "source": "Manual", "budget": { "spent": "4000.00", "limit": "5000.00", "percent":
80 } }
```

Chart data (GET /reports/chart-data/?kind=pie&from=2026-09-01&to=2026-09-30):

```
{ "kind": "pie", "labels": ["Food", "Rent", "Travel"], "values": [1000.00, 3000.00,
1000.00], "total": 5000.00 }
```

Import summary (200; the data shown on the summary page, written as JSON):

```
{ "imported": 7, "skipped": [{ "row": 4, "reason": "duplicate" }], "errors": [{ "row": 6,
"field": "amount", "message": "must be greater than 0" }] }
```

4.4 Error Handling, Logging & Monitoring

Errors are raised as typed exceptions in the service layer, translated to HTTP responses in one place (the controller layer), and shown to users as short messages that never reveal internals.

Principles

- Validate early: forms and services reject bad input before any database write.
- Fail safely: multi-row writes (CSV import, recurring run) are atomic, so a failure leaves no partial data.
- Be specific to the user, generic to the attacker: field errors name the field; login errors and permission errors are generic.
- Log for the team, not for the user: full detail goes to the log; the user sees a message and, for server errors, a reference ID.

Exception hierarchy

Exception	Raised by	Mapped to
`ValidationError` (field-level)	Forms, services	422 with inline field errors
`FileError`	Import Service	400 with reason
`PermissionDenied`	Security layer, views	403
`NotFound` (also for resources owned by another user)	Services	404
`AuthError`, `AccountLocked`	Auth Service	Login page with generic error (`AuthError`, 200); 429 (`AccountLocked`)
`ExternalServiceError`	Bank Sandbox Adapter	503 with retry message
Unhandled exception	Any	500 custom page with reference ID

Error scenarios and handling

Scenario	Requirement	Detection	Handling	User sees	Logged
Missing, disabled or wrong-type category on save	FR-007, FR-021	Transaction Service	Reject, no write	"Choose an active category of the right type"	No
Amount zero, negative or not a number	FR-005	Form validator	Reject, no write	"Amount must be greater than 0"	No
Password below policy at registration	FR-001	Form validator	Reject, no write	"Password must have at least 8 characters, a letter and a digit"	No
Overlapping budget	FR-014	Budget Service, inside a database transaction	Reject, no write	"A budget for this category already covers this period"	No
Malformed CSV row	FR-010	Import Service	Skip row, continue, list in result	Row number and reason per bad row	Count only
Wrong CSV header, bad encoding, over 2 MB	SEC-003	Import Service	Reject whole file before parsing rows	"File format not supported" with reason	Yes, warning
Import fails midway (database error)	NFR-004	Atomic transaction	Roll back all rows	"Import failed, nothing was saved. Try again."	Yes, error
Duplicate CSV row (in database or earlier in the same file)	FR-011	Import Service	Skip, list in result	"Skipped as duplicate"	Count only
Not logged in or expired session	SEC-004	Middleware	Redirect to login (401 for JSON); after login return to the original page	Login page	Yes, info
Wrong password	SEC-006	Auth Service	Generic error, count the attempt	"Email or password is incorrect"	Yes, warning
Five failed logins in 10 minutes	SEC-006	Auth Service	Lock the account for 10 minutes, respond 429	"Too many attempts, try again later"	Yes, warning

Scenario	Requirement	Detection	Handling	User sees	Logged
Customer opens an admin URL	SEC-002	Role decorator	403, no action	"You do not have access to this page"	Yes, warning
Another user's resource ID	Confidentiality	Owner-scoped query	404, no action	"Not found"	Yes, warning
Sandbox API down or times out (10 s)	FR-023	Bank Sandbox Adapter	Save nothing, return error	"Bank sync is unavailable, try later"	Yes, error
Recurring rule's category now disabled	FR-021, FR-024	Scheduler Service	Skip the rule, mark it for attention, continue with other rules	Notice on the recurring page	Yes, warning
Alert evaluation fails after a saved transaction	FR-015	Alert Service	Keep the transaction, log the error; evaluation runs again on the next transaction saved in that category	Nothing (transaction saved)	Yes, error
Unexpected server error	NFR-002	Global handler	500 page, rollback current request	"Something went wrong. Reference: ID"	Yes, error with stack trace

Implementation notes and monitoring

- HTMX requests receive errors as an HTML fragment with status 422 and the response is swapped into the form's error area (the page sets `htmx:responseError` and 4xx swapping).
- `DEBUG=False` in staging and production, with custom 403, 404 and 500 templates.
- Loggers: `security` (authentication and access events), `app` (business errors) and `audit` (admin changes), written to rotating files with no passwords, tokens or full CSV contents.
- The Scheduler Service processes each rule in its own database transaction so one failing rule does not block the others.
- Monitoring: the staging health URL is polled every 5 minutes to track NFR-002 availability (see Test Plan TC-AVAIL-01); the security logger is the source for the SEC-006 audit trail.

4.5 UX Design

The UI is server-rendered Django templates with HTMX partial updates, so most actions (adding a transaction, marking an alert read) refresh only the affected fragment instead of the full page. Design choices follow directly from the SRS:

- The transaction-entry flow is designed to satisfy NFR-003: a first-time user can record an expense from the dashboard in 3 primary screens or fewer, with inline validation errors next to the field that failed.
- Budget status is shown as a progress bar that changes color at the alert threshold (SRS `threshold_pct`, default 80%) and again when spend exceeds the limit, so the alerts in FR-015/FR-016 are visible without opening a separate page.

- Reports and the dashboard (FR-017, FR-018) use the same chart data endpoint so the pie, bar and line views stay consistent.
- Pages target the latest Chrome, Edge and Firefox plus one mobile browser (SRS Appendix E.5 / Test Plan Section 6); layout is responsive rather than a separate mobile app.
- Error and empty states (no transactions yet, empty report month) always render a message rather than a blank page.

4.6 Open Issues & Next Steps

- Real bank account linking beyond the sandbox adapter is deferred, as in SRS Section 1.2; the Adapter pattern (Section 3.5) is the extension point if it is added later.
- Multi-currency and multi-user/shared budgets are out of scope for this release (SRS 1.2) and are not designed here. Account deletion is likewise not offered in this release.
- Possible future enhancements noted but not designed: push/email notifications for alerts, biometric login, a dedicated mobile client.

5. Appendices

5.1 Glossary

Term	Meaning
MVT	Model-View-Template, Django's variant of MVC
Service layer	Plain Python modules holding business rules, called by views
Adapter	A class that hides an external service behind a fixed interface
HTMX	Library for partial page updates using HTML fragments over HTTP
DTO	Data transfer object: the simple data structure passed between layers
FR / NFR / SEC	Functional, non-functional and security requirement IDs in the SRS
RTM	Requirements traceability matrix
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (threat model categories)

5.2 References

- G-15 Software Requirements Specification v1.2 (Deliverable 2) and Synopsis (Deliverable 1).
- G-15 Test Plan v1.1 (companion document, IEEE 829 format).
- IEEE Std 1016-2009, Software Design Descriptions.
- IEEE Std 42010, Architecture Description.
- OWASP Top 10 (2021).
- Django documentation (MVT, ORM, authentication, CSRF).

5.3 Tools

Mermaid.js and PlantUML for the UML diagrams in this document (sequence, class and component diagrams); Git and GitHub for version control and the GitHub Actions CI pipeline; Pytest, Coverage.py, Locust and OWASP ZAP for the verification referenced from the Test Plan.

5.4 Key Classes

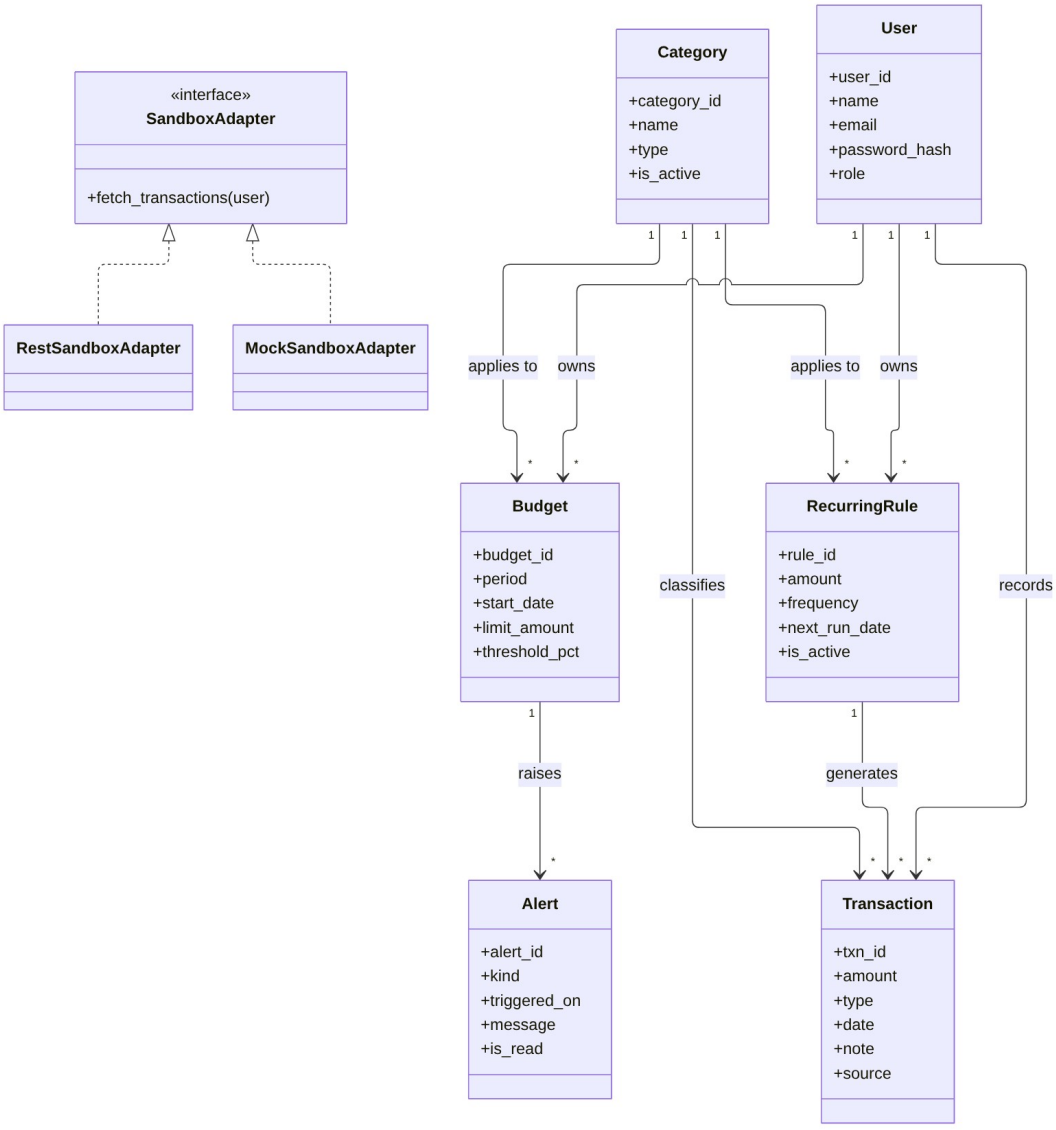


Figure 8: Key classes (UML class diagram)

5.5 Design Decisions

Decision	Reason
Server-rendered pages with HTMX instead of a separate single-page front end	Fewer moving parts and one language for a small team; matches the SRS tool stack
Session authentication, not tokens	Django sessions fit server-rendered pages and give server-side logout (FR-004)

Decision	Reason
Alert evaluated synchronously after each save	Alerts appear immediately (FR-013 to FR-016); load is small for a single-user app
Money stored as `DecimalField`, never float	Avoids rounding errors in budget comparisons and boundary tests
Import is idempotent through a duplicate check (database lookup, in-file key set, and a CSV-only unique index as safety net)	Satisfies NFR-004 whatever the file order or number of retries, without blocking identical manual or recurring transactions
Sandbox behind an interface with a mock	Lets core logic be tested with no external service (NFR-006)

5.6 Deployment and Data View

The system deploys as one web application process plus a daily scheduled job, both sharing one database.

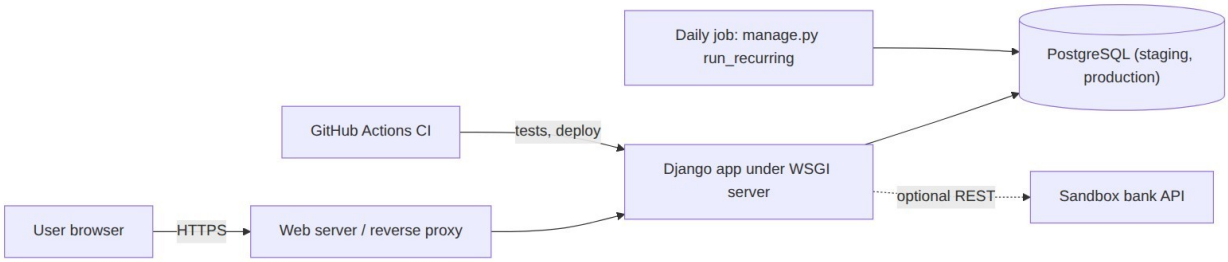


Figure 9: Deployment view

Development uses SQLite and the Django development server. The daily job calls `run_due_rules(today)` from the Scheduler Service, so scheduler logic is the same code the tests call directly (FR-024).

The entities and fields are those of SRS Section 7.2 and Appendix B. Constraints and indexes below enforce the SRS rules.

Entity	Constraint or index	Purpose
User	Unique index on email	FR-001
Category	type limited to Expense or Income; `is_active` default true	FR-020, FR-021
Transaction	Check `amount > 0`; index on (`user_id`, date); index on (`user_id`, `category_id`, date); note stored as empty string when omitted	FR-005, NFR-001
Transaction	Partial unique index on (`user_id`, date, amount, note) where source = CSV; applies to CSV-imported rows only, so identical manual, recurring or sandbox transactions stay allowed. The Import Service also checks existing transactions of every source and earlier rows of the same file before inserting	FR-011, NFR-004
Budget	Check `limit_amount > 0` and `threshold_pct` between 1 and 100; overlap checked in the service inside a transaction using the period rule of SRS Appendix B.3	FR-012, FR-014
RecurringRule	Check `amount > 0`; index on `next_run_date`	FR-019, FR-024
Alert	Index on (`budget_id`, `triggered_on`); deleted with its budget	FR-012, FR-015, FR-016

5.7 Design-to-Test Traceability

Design element	Requirements	Test cases (see Test Plan)
4.2 Login sequence	FR-002, SEC-004, SEC-006	TC-AUTH-02, TC-SEC-04, TC-SEC-06
4.2 Add expense and alert sequence	FR-005, FR-007, FR-013, FR-015, FR-016	TC-TXN-01, TC-TXN-03, TC-BUD-02, TC-BUD-04, TC-BUD-05
4.2 CSV import sequence	FR-009, FR-010, FR-011, NFR-004, SEC-003	TC-TXN-05, TC-TXN-07, TC-REL-01, TC-SEC-03
4.2 Recurring generation sequence	FR-019, FR-024	TC-REC-01, TC-REC-02
Security architecture (Section 3.9)	SEC-001 to SEC-006	TC-SEC-01 to TC-SEC-07
Error handling (Section 4.4)	FR-007, FR-010, FR-014, FR-023	TC-TXN-03, TC-TXN-05, TC-BUD-03, TC-TXN-06
Report queries and indexes (5.6)	NFR-001	TC-PERF-01